

Projeto da Disciplina de Engenharia de Software

Objetivo: Utilizar conceitos fundamentais da disciplina, relacionados às principais práticas de Engenharia de Software, tais como processos de software, metodologias ágeis, gestão de projetos de software, engenharia de requisitos, prototipagem, projeto e desenvolvimento.

Entrega 01 (conforme data anteriormente definida)

Documento contendo:

1. Definição do Escopo: Descrição de alto nível sobre o software, clientes, usuários (*stakeholders*), e sobre o problema específico a ser resolvido. O escopo do projeto é livre, devendo ser definido pela equipe. Seria interessante que o projeto contasse com usuários reais que fornecessem suas necessidades reais (histórias de usuário). Caso não seja possível, a equipe deve propor um escopo de um potencial produto.
2. Apresentação do Backlog do Produto, contendo as histórias de usuário que descrevem os requisitos funcionais do software (necessidades dos usuários envolvidos) – requisitos de usuário.
3. Planejamento do desenvolvimento em três Sprints com cerca de 3 a 4 semanas cada, apresentando o Backlog de cada uma das Sprints, e as respectivas tarefas (tasks) relacionadas ao desenvolvimento de cada história de usuário. Utilizar alguma ferramenta de gestão de projetos ágeis como Trello ou Jira para a gestão do planejamento e divisão de tarefas. Inserir no relatório *screenshots* (*prints* de tela) do uso da ferramenta escolhida. Fornecer link, usuário e senha para acesso ao projeto na ferramenta de gestão.
4. Descrição de casos de uso (cenários de uso) considerando cada história de usuário (detalhamento mais técnico das histórias de usuário) – requisitos de usuário. Veja **Anexo I** sobre Descrição de Casos de Uso. **Apresentar também o Diagrama de Casos de Uso** mostrando os atores e relacionamentos, e respectivas dependências entre os casos de uso (inclusão, extensão, generalização).
5. Modelagem conceitual (diagrama entidade-relacionamento) completo referente ao produto a ser desenvolvido.
6. Protótipo não funcional (protótipo de baixa fidelidade), que é um rascunho inicial do sistema, contemplando telas (interface de usuário), sem funcionalidades implementadas, mas que sirva como forma de validar os requisitos levantados inicialmente junto aos usuários (Veja **Anexo II** sobre Prototipação).

Entrega 02 (conforme data anteriormente definida)

Documento contendo:

1. Modelo de classes (projeto), contemplando adequadamente as classes, relacionamentos, atributos e os métodos para implementação dos requisitos funcionais especificados (histórias de usuário) em

relação ao modelo conceitual apresentado. Descrever qual ou quais métodos implementam cada história de usuário (**rastreabilidade de requisitos**).

2. Produto final, sendo este um Produto Mínimo Viável (MVP - Minimum Viable Product). Pode-se utilizar plataformas No-Code ou Low-Code para seu desenvolvimento. Veja **Anexo III** para mais informações sobre MVP e No-Code. Qualquer tecnologia e linguagem podem ser utilizadas.

2.1 Fornecer no documento o link para vídeo apresentando a versão MVP desenvolvida (protótipo utilizável de alta-fidelidade)

2.2 Fornecer link de acesso para o projeto (código fonte) no git ou respectiva plataforma low-code em que foi desenvolvido.

Sugere-se para esta atividade a utilização do Bubble.io. Mas, qualquer outra ferramenta/linguagem pode ser utilizada.

Anexo I - Observações sobre Descrição de Caso de Uso

Inclui **duas listas de passos**:

A primeira representa o **fluxo normal de passos** necessários para concluir uma operação com sucesso. Ou seja, o fluxo normal descreve um cenário onde **tudo dá certo**, às vezes chamado também de **“fluxo feliz”**.

A segunda lista inclui extensões ao fluxo normal. Representam **alternativas de execução de um passo normal** ou então **situações de erro**. Ambos os fluxos — normal e extensões — serão posteriormente implementados no sistema.

Exemplo:

Transferir Valores entre Contas (UC002)

Ator: Cliente do Banco

Fluxo normal:

- 1 - Incluir o caso de uso **“autenticar cliente”** ou UC001.
- 2 - Cliente informa agência e conta de destino da transferência
- 3 - Ciente informa valor que deseja transferir
- 4 - Cliente informa a data em que pretende realizar a operação. Data informada deve ser a data

atual ou no máximo um ano a frente.

5 - Sistema efetua transferência

6 - Sistema pergunta se o cliente deseja realizar uma nova transferência

Extensões:

2a - Se conta e agência incorretas, solicitar nova conta e agência

3a - Se valor acima do saldo atual, solicitar novo valor

4a - Se a data não for válida, solicitar nova data

5a - Se data informada é a data atual, transferir imediatamente

5b - Se data informada é uma data futura, agendar transferência

O passo 1 do fluxo normal **inclui o caso de uso “autenticar cliente”**.

A sintaxe para tratar inclusões (reúso) é simples: menciona-se o nome do caso de uso a ser incluído, que deve estar sublinhado.

A semântica também é clara: todos os passos do caso de uso incluído devem ser executados antes de prosseguir.

Extensões, as quais têm dois objetivos:

Detalhar **condicionais de** passos do fluxo normal.

Usamos extensões para especificar que a transferência deve ser imediatamente realizada **se a data informada for a data corrente (extensão 5a)**.

Caso contrário, temos um agendamento de transferência, que vai ocorrer na data futura que foi informada (extensão 5b).

Tratar erros, exceções, cancelamentos, etc.

Extensão para especificar que um novo valor deve ser solicitado, caso não exista saldo suficiente para a transferência (extensão 3a).

Devido à existência de fluxos de extensão, recomenda-se **evitar comandos de decisão (“se”) no fluxo normal de casos de uso**.

Descrições de casos de uso podem incluir seções adicionais, tais como:

- (1) propósito do caso de uso;
- (2) pré-condições, isto é, o que deve ser verdadeiro antes da execução do caso de uso;
- (3) pós-condições, isto é, o que deve ser verdadeiro após a execução do caso de uso; e
- (4) uma lista de casos de uso relacionados.

Boas práticas de escrita de casos de uso:

As ações de um caso de uso devem ser escritas em uma linguagem simples e direta:

<u>sujeito das ações, seguido de um verbo.</u>
--

Por exemplo, “o cliente insere o cartão no caixa eletrônico”.

Porém, se a ação for realizada pelo sistema, escreva algo como: “o sistema valida o cartão inserido”.

Casos de uso não são algoritmos escritos em pseudo-código.

Usuários do sistema cujos requisitos estão sendo documentados **devem ser capazes de ler, entender e descobrir problemas em casos de uso.**

Casos de uso **não devem tratar de aspectos tecnológicos ou de design.**

Além disso, eles **não precisam mencionar a interface que o ator principal** usará para se comunicar com o sistema.

Por exemplo, não se deve escrever algo como: **“o cliente pressiona o botão verde para confirmar a transferência”.**

Na fase de documentação de requisitos decisões sobre tecnologia, design, arquitetura e interface com o usuário ainda não estão em nosso radar.

O objetivo deve ser documentar “o que” o sistema deverá fazer e não “como” ele irá implementar os requisitos especificados.

Anexo II – Prototipação ou Prototipagem

Uma das ferramentas importantes da engenharia de requisitos é a validação de requisitos, confirmando junto aos usuários (stakeholders) que há um entendimento comum sobre o sistema que deve ser desenvolvido.

Um protótipo é um rascunho inicial do sistema, contemplando telas (interface de usuário), sem funcionalidades implementadas, mas que sirva como forma de validar os requisitos levantados inicialmente junto aos usuários.

Um protótipo é uma versão inicial de um sistema de software, usado para demonstrar conceitos, experimentar opções de projeto e descobrir mais sobre o problema e suas possíveis soluções. O desenvolvimento rápido e iterativo do protótipo é essencial para que os custos sejam controlados e os stakeholders do sistema possam experimentá-lo no início do processo de software.

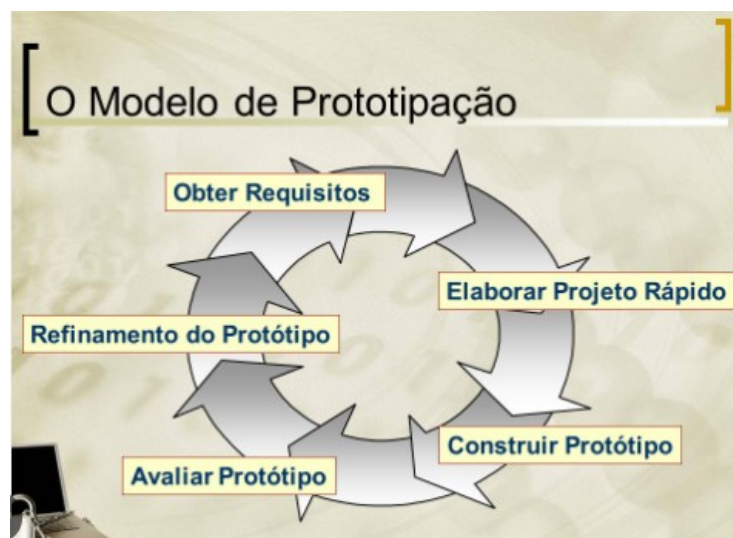
Um protótipo de software pode ser usado em um processo de desenvolvimento de software para ajudar a antecipar as mudanças que podem ser requisitadas:

1. No processo de engenharia de requisitos, um protótipo pode ajudar na elicitación e validação de requisitos de sistema.
2. No processo de projeto de sistema, um protótipo pode ser usado para estudar soluções específicas do software e para apoiar o projeto de interface de usuário. Protótipos do sistema permitem aos usuários ver quão bem o sistema dá suporte a seu trabalho. Eles podem obter novas ideias para requisitos e encontrar pontos fortes e fracos do software; podem, então, propor novos requisitos do sistema. Além disso, o desenvolvimento do protótipo pode revelar erros e omissões nos requisitos propostos.

Neste contexto, a **prototipação (conjunto de telas, interface de usuário)** é um artifício importante para a validação dos requisitos, pois, além de servir como um primeiro rascunho de um produto ou serviço, tem como objetivo amadurecer ideias e engajar pessoas no processo de criação. Esta etapa impacta diretamente na produtividade de toda a equipe e gera valor ao cliente. É nesta fase que as ideias são colocadas em prática para facilitar o entendimento de uma aplicação ou sistema.

Além do baixo custo, a **prototipação no desenvolvimento de software (conjunto de telas, interface de usuário)** tem como característica reduzir os riscos e permitir que todas as validações sejam feitas antes da implementação. De acordo com Jakob Nielsen, cientista da computação e cofundador da Nielsen Norman Group, “estima-se que seja 100 vezes mais barato fazer mudanças antes de escrever qualquer código, do que aplicá-las após a implementação”.

O ciclo de prototipação é apresentado a seguir:



Para isto pode-se utilizar qualquer ferramenta de prototipação, por exemplo, o **Figma** é uma das

ferramentas mais utilizadas no mercado para prototipagem. Sendo uma ferramenta on-line, não há necessidade de downloads e você pode acessar o Figma em computadores Windows, Mac e Linux.

Outras ferramentas para prototipagem rápida, além do Figma:

Gimp;

Inkscape;

Da Vinci;

Blender;

Krita;

Scribus;

Canva;

Outras...

O protótipo é um documento essencial, que permite que desenvolvedores e usuários cheguem a um acordo sobre o que é realmente para ser desenvolvido.

Anexo III – MVP e No-Code

Produto Mínimo Viável (MVP - Minimum Viable Product)

O maior desperdício que pode existir é passar anos levantando requisitos e implementando um sistema que depois não vai ser usado, pois ele resolve um problema que não interessa mais a seus usuários. Portanto, se é para um sistema falhar — por não ter sucesso, usuários ou mercado — é melhor falhar rapidamente, pois o desperdício de recursos será menor.

Um sistema simples, com um conjunto de requisitos mínimos, mas que sejam suficientes para testar a viabilidade de continuar investindo no seu desenvolvimento. Em Lean Startup, esse primeiro sistema é chamado de Produto Mínimo Viável (MVP). Às vezes, costuma-se dizer também que o objetivo de um MVP é testar uma hipótese de negócio. Justamente por causa dessa incerteza extrema, é preciso aprender. Então o lean startup é um método para aprender o mais rápido possível, através de experimentos, o que deve ser desenvolvido (o que os clientes desejam).

Low-Code e No-Code

São abordagens de desenvolvimento de software que permitem a criação de aplicações com pouco ou nenhum código manual. As plataformas Low-Code oferecem interfaces visuais para construir aplicações, enquanto as plataformas No-Code eliminam completamente a necessidade de

codificação, permitindo que até mesmo usuários sem conhecimentos técnicos possam desenvolver software. Essas abordagens democratizam o desenvolvimento de software, permitindo que mais pessoas participem do processo de criação de aplicações.

Uma das maiores vantagens das plataformas Low-Code e No-Code é a velocidade com que as aplicações podem ser desenvolvidas. As interfaces visuais permitem que desenvolvedores e não desenvolvedores criem rapidamente protótipos e aplicações completas.

Exemplo: Uma equipe de marketing pode usar uma plataforma No-Code como Bubble para criar um portal de clientes em dias, em vez de semanas ou meses, se utilizassem uma abordagem tradicional de desenvolvimento. 2. Redução de Custos A necessidade reduzida de desenvolvedores altamente especializados pode resultar em economia de custos significativos.

Além disso, a rápida criação e implantação de aplicações pode economizar tempo e recursos.

Exemplo: Uma pequena empresa pode evitar os altos custos de contratar uma equipe de desenvolvimento completa, utilizando ferramentas No-Code para criar suas aplicações internas.

Startups e MVPs Startups frequentemente utilizam plataformas No-Code para criar seus MVPs (Minimum Viable Products). A capacidade de desenvolver rapidamente uma aplicação funcional permite que as startups testem suas ideias no mercado com rapidez e eficiência. Exemplo: Uma startup pode usar Bubble para construir rapidamente um protótipo funcional de sua plataforma de marketplace, testar com usuários reais e iterar com base no feedback.

Exemplo: Uma startup pode usar Bubble para construir rapidamente um protótipo funcional (utilizável) de sua plataforma de marketplace, testar com usuários reais e iterar com base no feedback.

Empresas de todos os tamanhos utilizam plataformas Low-Code e No-Code para automatizar processos internos. Isso pode incluir desde o gerenciamento de inventário até a automação de fluxos de trabalho de recursos humanos.

Algumas das melhores plataformas Low-Code e No-Code atualmente:

- Bubble.io,
- Glide,
- Kodular,
- Thunkable

Veja o comparativo destas plataformas em

<https://www.semcodar.com.br/como-escolher-um-plano-gratuito-ou-pago-bubble-io-glide-kodular-thunkable/>

Bubble.io

Bubble é uma plataforma No-Code popular entre startups e pequenas empresas. Lançada em 2012, Bubble permite que os usuários criem aplicações web totalmente funcionais sem escrever uma única linha de código.

A plataforma oferece uma interface intuitiva de arrastar e soltar, juntamente com uma linguagem visual para definir a lógica da aplicação.

Funcionalidades:

- Editor Visual: Interface de usuário intuitiva para design de páginas web e lógica de negócios.
 - Banco de Dados Integrado: Ferramentas integradas para gerenciar dados da aplicação.
 - Escalabilidade: Suporte para aumentar a capacidade de acordo com a demanda da aplicação.
-